scripting_guide_and_examples（脚本指南与示例）

未发布作品。© 2023 Siemens 本材料包含 Siemens Industry Software, Inc.、其子公司或其关联公司（统称为"Siemens"）或其许可方的商业秘密或其他机密信息。对这些信息的访问和使用严格受限于客户与 Siemens 签订的适用协议中的规定。未经 Siemens 明确书面许可，不得在客户设施之外复制、分发或以其他方式披露本材料，也不得以 Siemens 未明确授权的任何方式使用本材料。 本文件仅供参考和指导之用。Siemens 保留随时更改本出版物中包含的规格和其他信息的权利，读者在所有情况下都应咨询 Siemens 以确认是否有任何变更。本文件中关于产品、特性或功能的陈述仅构成技术信息，不构成任何保证或担保，也不应引起 Siemens 的任何责任。 Siemens 不承担所有保证，包括但不限于适销性和特定用途适用性的默示保证。特别是，Siemens 不保证产品的运行不会中断或无错误。 管辖 Siemens 产品销售和许可的条款和条件在 Siemens 与其客户之间的书面协议中规定。Siemens 的最终用户许可协议和通用合同协议可在以下网址查看：https://www.sw.siemens.com/en-US/sw-terms/ 商标：本文使用的商标、徽标和服务标记（"Marks"）归 Siemens 或其他方所有。未经 Siemens 或适用的标记所有者事先书面同意，任何人不得使用这些标记。本文中第三方标记的使用并非试图将 Siemens 表示为产品的来源，而是旨在标识来自或关联于特定第三方的产品。Siemens 商标列表可在以下网址查看：https://www.plm.automation.siemens.com/global/en/legal/trademarks.html。 注册商标 Linux® 是根据 LMI 的分许可使用的，LMI 是 Linus Torvalds 的独家被许可方，后者是该标记在全球范围内的所有者。

关于 Siemens Digital Industries Software Siemens Digital Industries Software 是全球领先的全生命周期管理（PLM）软件和服务提供商，拥有 700 万个许可席位和全球 71,000 家客户。总部位于德克萨斯州普莱诺，Siemens Digital Industries Software 与企业协作，提供开放式解决方案，帮助他们将更多创意转化为成功的产品。有关 Siemens Digital Industries Software 产品和服务的更多信息，请访问 https://www.siemens.com/plm 支持中心：https://www.support.sw.siemens.com 文档反馈：https://www.support.sw.siemens.com/doc_feedback_form

---

支持的语言与脚本设置

支持的语言

JavaScript

Polarion 2304

Polarion 22 R2

Polarion 21 R2 及更早版本

文档

Groovy

Polarion 22 R1

文档

Polarion 21 R2 及更早版本

Polarion SDK

脚本位置

工作流脚本

ScriptCondition（脚本条件）

脚本实现

可用对象

ScriptCondition 的开始

ScriptCondition 的结束

脚本配置

脚本调试

脚本示例

CheckDocumentCommentsStatus.js - 返回布尔值示例

CheckDocumentCommentsStatus.js - 返回字符串示例

ScriptFunction（脚本函数）

脚本实现

可用对象

ScriptFunction 的开始

ScriptFunction 的结束

脚本配置

脚本调试

throw 的使用

脚本示例

IncrementVersion.js - 使用 2 个配置参数和调试信息

CreateDocumentBaseline.js

作业脚本

脚本配置

脚本实现

可用对象

script.job 的内容

如何使用 scope

如何读取脚本属性

如何使用 workDir 变量

如何启动和提交事务

使用 doAsUser 以其他用户身份运行 script.job 的部分代码

脚本执行

手动执行 script.job

自动执行 script.job

script.job 的权限

脚本调试

脚本示例

emailDigestJob.js

FAQ（常见问题）

欢迎阅读 Polarion 脚本 SDK 指南。本文档面向需要为工作流条件、函数和作业编写自定义脚本的开发人员和其他人员。它涵盖了支持的脚本语言和版本、特定用例（含示例）、脚本配置、调试和实现。本指南最后附有一些常见问题解答（FAQ）。 如果您有自定义脚本需求，但缺乏必要的资源，Polarion 的专业服务团队在各种行业的自定义脚本以及许多其他 Polarion 自定义和集成方面拥有丰富而深入的经验。更多信息，请访问 https://polarion.plm.automation.siemens.com/training-and-consulting 或联系您公司的 Siemens 销售代表。 提示：从支持中心下载 SDK 脚本指南示例文件存档以获取更多脚本示例。

支持的语言与脚本设置

支持的语言

您可以使用 Groovy 或 JavaScript 语言为工作流和作业创建 Polarion 自定义脚本。

JavaScript

Polarion 2304

从 Polarion 2304 开始，默认的 JavaScript 引擎是 GraalVM。如果您有任何为 Polarion 早期版本编写的 Nashorn 或 Rhino 脚本，您需要将它们升级到 GraalVM。请参阅 GraalVM 迁移指南和本文档中的"FAQ"部分来适配您的脚本。

GraalVM 在同一 js 上下文中对多线程有限制。这意味着不能使用多线程。详情请参阅：https://www.graalvm.org/22.3/reference-manual/js/Multithreading/

Polarion 22 R2

Polarion 22 R2 是最后一个以 Nashorn 为默认 JavaScript 引擎的 Polarion 版本。从 2304 开始，应使用 GraalVM 引擎编写 JavaScript 脚本，因为它将成为未来 Polarion 版本中的默认引擎。要将 GraalVM 设为默认 JavaScript 引擎，请在 polarion.properties 文件中将以下属性设置为"true"：com.polarion.scripting.useGraalJsEngine=true。

GraalVM 在同一 js 上下文中对多线程有限制。这意味着不能使用多线程。详情请参阅：https://www.graalvm.org/22.3/reference-manual/js/Multithreading/

本指南中的所有 JavaScript 示例均使用 GraalVM 编写和测试。

Polarion 21 R2 及更早版本

Polarion 21 R2 及更早版本中使用的默认 JavaScript 引擎是 Nashorn。如果您有旧脚本，也可以通过在 polarion.properties 文件中将 com.polarion.scripting.useRhinoJsEngine 属性设置为"true"来启用 Rhino 作为默认引擎。

如果您正在迁移到 Polarion 22 R1 及更高版本，应审查您的脚本并将其适配到 GraalVM。Polarion 21 R2 是最后一个支持 Rhino 的版本。请参阅 GraalVM 迁移指南和本文档中的"FAQ"部分来适配您的脚本。

文档

GraalVM JavaScript 兼容性指南：https://www.graalvm.org/reference-manual/js/JavaScriptCompatibility/ 请注意，Polarion 在启用 GraalVM 时不使用 Nahshorn 兼容模式。

如果您想将为旧 JavaScript 引擎编写的脚本迁移到 GraalVM： - 从 Rhino 迁移指南：https://www.graalvm.org/reference-manual/js/RhinoMigrationGuide/ - 从 Nashorn 迁移指南：https://www.graalvm.org/reference-manual/js/NashornMigrationGuide/

Groovy

Polarion 22 R1

从 Polarion 22 R1 开始，支持使用 Groovy v2.4.21 编写的脚本。

本指南中的所有 Groovy 示例均使用 Groovy v2.4.21 测试。

文档

Groovy v2.4.21 官方文档可以在以下位置找到： https://docs.groovy-lang.org/docs/groovy-2.4.21/html/documentation/

Polarion 21 R2 及更早版本

Polarion 21 R2 及更早版本支持使用 Groovy v1.5.7 编写的脚本。

Polarion SDK

您可以在安装文件系统的 $POLARION_HOME$/polarion/sdk 文件夹中找到 Polarion SDK 内容。文档位于 doc 子文件夹中。 [POLARION_HOME]/polarion/sdk/index.html 文件包含指向所有 SDK 资源的链接。您可以通过在 Polarion 服务器 URL 后附加 polarion/sdk/ 来通过 Web 浏览器访问 SDK 索引页。

脚本位置

脚本必须存储在 scripts 目录中：（如果该目录尚不存在，请创建它。） - Windows：C:\Polarion\scripts - Linux：/opt/polarion/scripts

注意：以上根路径为 Polarion 安装默认路径。安装管理员可以指定不同的安装根目录。

工作流脚本

您可以使用自定义脚本扩展 Work Item（工作项）、Document（文档）和 Test Run（测试运行）的工作流。这些脚本可以在允许工作流操作之前检查是否满足特定条件（例如，是否仍有未解决的评论），或者在执行工作流操作时执行某个函数（例如，递增文档的版本字段）。

ScriptCondition（脚本条件）

ScriptCondition（脚本条件）是一种用于在允许工作流操作之前检查是否满足特定条件的脚本类型。当用户在 Polarion UI 中发起对 Status（状态）字段的更改时，Polarion 会执行与 ScriptCondition 关联的脚本，并生成用户可以采取的可能操作列表。

例如，假设一个文档工作流有一个从"审核中"到"已审核"的转换，该转换配置了一个条件脚本来检查文档是否没有任何未解决的评论。如果是这样，那么当文档有未解决的评论时，Polarion 会禁用该转换操作：

脚本实现

ScriptCondition（脚本条件）是一个返回布尔值（true/false）的脚本，用于向 Polarion 指示条件是否满足。脚本也可以返回一个字符串，Polarion 随后会将其显示给用户作为不允许转换的原因。（返回字符串等同于返回 false。）

例如，如果仍有未解决的评论，脚本返回的字符串为：
`Unresolved comment found. Make sure to resolve all of your comments.`

可用对象

ScriptCondition 脚本可以访问以下 Polarion Open API 接口：

- `com.polarion.alm.tracker.workflow.ICallContext`（用于获取执行工作流的工作项、文档或测试运行对象。）
- `com.polarion.alm.tracker.workflow.IArguments`（用于获取脚本的自定义配置设置。）

在脚本中，您可以访问以下变量：

- `workflowContext`：ICallContext 类型的对象
- `arguments`：IArguments 类型的对象

ScriptCondition 的开始

通常，ScriptCondition 的开始是获取执行工作流脚本的对象。这是通过使用 workflowContext 对象并执行以下代码完成的：

JavaScript:

```
var baseObject = workflowContext.getTarget();
```

Groovy:

```
def baseObject = workflowContext.getTarget();
```

根据工作流是从 Work Item（工作项）、Document（文档）还是 Test Run（测试运行）执行的，workflowContext.getTarget() 调用可以返回以下对象类型：

- `com.polarion.alm.tracker.model.IWorkItem`（如果从工作项执行。）
- `com.polarion.alm.tracker.model.IModule`（如果从文档执行。）
- `com.polarion.alm.tracker.model.ITestRun`（如果从测试运行执行。）

如果您在脚本配置（参见下一节）中使用自定义配置参数，则可以使用 arguments 对象访问它们。 例如，如果您有一个名为 versionFieldName 的自定义参数，则可以通过执行以下代码访问其值：

JavaScript:

```
// 获取在工作流中配置的版本字段名称。如果配置中未提供值，默认值将为"version"
var versionFieldID = arguments.getAsString("versionFieldName", "version");
```

Groovy:

```
// 获取在工作流中配置的版本字段名称。如果配置中未提供值，默认值将为"version")
def versionFieldID = arguments.getAsString("versionFieldName", "version");
```

ScriptCondition 的结束

ScriptCondition 必须以布尔值或字符串结束。这告诉 Polarion 是否启用脚本在其上执行的工作流操作。

请注意，您不能在脚本末尾使用"return"语句。为了让 Polarion 理解布尔值，脚本的最后一个语句必须直接是布尔值或字符串。

例如，如果您正在测试 Severity（严重性）字段值是否为"should_have"，则脚本的最后一行应该是：

JavaScript/Groovy:

```
baseObject.getSeverity().getId().equals("should_have");
```

在某些情况下，您也可以将布尔值存储在变量中，然后在脚本末尾添加它：

JavaScript/Groovy:

```
if(ABC)
{
    (...)
    myResult = true;
}
else
{
    myResult = false;
}
// 脚本的最后一行
myResult;
```

如果您想向用户显示消息，则可以返回字符串而不是布尔值。如果返回字符串，Polarion 会将其视为布尔值"False"。

JavaScript/Groovy:

```
if(ABC)
{
    (...)
    myResult = true;
}
else
{
    myResult = "Unresolved comment found. Make sure to resolve all of your
comments.";
}
// 脚本的最后一行
myResult;
```

脚本配置

步骤 1：根据您希望配置的工作流类型，进入以下管理页面之一： - 工作项：Administration → Work Items → Workflow - 文档：Administration → Documents & Pages → Document Workflow - 测试运行：Administration → Testing → Test Run Workflow

步骤 2：点击您要添加脚本条件的制品旁边的 Edit（编辑）。 步骤 3：点击您要添加脚本条件的操作旁边的 Edit（编辑）。 步骤 4：从 Conditions（条件）下拉列表中选择 ScriptCondition，然后点击添加。 步骤 5：点击新创建的 Condition 旁边的 Edit（编辑）。 步骤 6：定义以下必需参数： - engine：要使用的脚本引擎名称。支持 Groovy（.groovy）和 JavaScript/ECMAScript（.js）文件类型。有效值：groovy 表示 Groovy 引擎，js 表示 JavaScript/ECMAScript。 - script：scripts 文件夹中要作为条件运行的脚本名称。

例如，一个名为 CheckDocumentCommentsStatus.js 的 JavaScript：

如果您使用自定义参数，只需添加新参数，其中 Name 是自定义参数的名称，Value 是在脚本中读取的值。 假设脚本检查是否有超过五个未解决的评论，您希望通过 numberOfComments 参数使此数字可配置。 在脚本中，以下代码用于读取参数的值：

JavaScript:

```
// 获取在工作流中配置的评论数。如果配置中未提供值，默认值将为 0)
var numberComments = arguments.getAsInteger("numberOfComments", 0);
```

Groovy:

```
// 获取在工作流中配置的评论数。如果配置中未提供值，默认值将为 0)
def numberComments = arguments.getAsInteger("numberOfComments", 0);
```

步骤 7：点击关闭按钮关闭 Parameters（参数）对话框，再次点击关闭 Details for Action（操作详情）对话框。 新的 Condition（条件）将出现在您为其定义的操作中。

步骤 8：点击顶部的 Save（保存）。

脚本调试

ScriptCondition 没有可用的调试器。通常使用的调试方法是通过自定义日志文件，可以从 Polarion 日志文件夹读取这些文件。 路径：C:\Polarion\data\main\logs（Windows）和 /opt/polarion/data/main/logs（Linux）。 其思路是让脚本创建一个日志文件，用于显示脚本中变量的内容。要使日志文件填充您的日志条目，您必须执行最终用户在 Polarion UI 中会执行的操作以触发 ScriptCondition 的执行，即点击工作项、文档或测试运行的 Status（状态）字段并选择您的脚本所对应的工作流操作。

要为脚本创建日志文件，您可以执行：

JavaScript:

```
// 创建日志文件
var FileWriter = Java.type('java.io.FileWriter');
var outFile = new FileWriter("./logs/main/NAME_OF_YOUR_SCRIPT.log");
var BufferedWriter = Java.type('java.io.BufferedWriter');
```

```
var out = new BufferedWriter(outFile);
```

Groovy:

```
// 创建日志文件
def logFile = new File('./logs/main/NAME_OF_YOUR_SCRIPT.log').newOutputStream();
```

然后在脚本中，您可以简单地为任何您想跟踪的内容创建日志条目：

JavaScript:

```
out.write("Running NAME_OF_YOUR_SCRIPT.js script"); out.newLine(); out.flush();
```

Groovy:

```
logFile << "Running NAME_OF_YOUR_SCRIPT.groovy script \n";
```

在脚本结束时，务必始终关闭日志文件：

JavaScript:

```
// 关闭日志文件
out.close();
```

Groovy:

```
// 关闭日志文件
logFile.close();
```

脚本示例

CheckDocumentCommentsStatus.js - 返回布尔值示例

此示例在找到至少一个未解决的文档评论时将返回 false。

```
// 当未找到评论时，脚本将返回 true
// -------------------------------------------------------
var returnValue = true;

// 从工作流上下文构造函数实例化文档对象
// ----------------------------------------------------------------------
var document = workflowContext.getTarget();

// 检索文档中的所有评论
// 参见：com.polarion.alm.tracker.model.IModule 获取所有文档方法列表
//
-------------------------------------------------------------------------------
var allComments = null;
allComments = document.getComments();

// 检查从文档中检索的每个评论的状态
// ------------------------------------------------------------
for (i = 0; i < allComments.size(); i++)
{
  var comment = allComments.get(i);
  // 检查评论是否未解决
  // 参见：com.polarion.alm.tracker.model.IModuleComment
  // 获取所有文档评论方法列表
  // ------------------------------------------------
  if(!comment.isResolvedComment())
  {
    // 一旦找到未解决的评论，我们将返回 FALSE
    // ------------------------------------------------------------------
    returnValue = false;
    break;
  }
}

// 脚本最后一行是布尔值。不会向用户显示任何消息
// -------------------------------------------------------------------------------
returnValue;
```

CheckDocumentCommentsStatus.js - 返回字符串示例

此示例在找到至少一个未解决的文档评论时将向用户返回一个字符串。

```
// 跟踪是否找到未解决评论的变量
// -------------------------------------------------------
var isUnresolvedCommentFound = false;

// 从工作流上下文构造函数实例化文档对象
// ---------------------------------------------------------------------
```

```
var document = workflowContext.getTarget();

// 检索文档中的所有评论
// 参见：com.polarion.alm.tracker.model.IModule 获取所有文档方法列表
//
-------------------------------------------------------------------------------
var allComments = null;
allComments = document.getComments();

// 检查从文档中检索的每个评论的状态
// ------------------------------------------------------------
for (i = 0; i < allComments.size(); i++)
{
  var comment = allComments.get(i);
  // 检查评论是否未解决
  // 参见：com.polarion.alm.tracker.model.IModuleComment
  // 获取所有文档评论方法列表
  // ---------------------------------------------------
  if(!comment.isResolvedComment())
  {
    // 一旦找到未解决的评论，我们将变量设置为 true
    // ---------------------------------------------------------------------
    isUnresolvedCommentFound = true;
    break;
  }
}

// 当未找到评论时，脚本将返回 true
// -------------------------------------------------------
var returnValue = true;
if (isUnresolvedCommentFound)
{
  returnValue = "Unresolved comment found. Make sure to resolve all your comments"
}

// 脚本最后一行在未找到评论时将返回 true，工作流操作将被启用。
// 当找到未解决的评论时，它将返回一个显示给用户的字符串，并禁用工作流操作
//
-------------------------------------------------------------------------------
-
returnValue;
```

ScriptFunction（脚本函数）

ScriptFunction（脚本函数）是一种脚本类型，用于在用户更改工作项、文档或测试运行的 Status（状态）字段从而发起工作流操作时执行一个或多个函数。Polarion 将在 Save（保存）操作期间调用与 ScriptFunction 关联的脚本，该操作在用户通过工作流操作更改了工作流的 Status（状态）之后执行。

例如，如果文档工作流有一个从 Approved（已批准）到 Draft（草稿）的转换，该转换配置了一个脚本函数，用于递增存储在特定文档自定义字段中的版本号，那么当用户执行工作流操作并点击文档上的 Save（保存）时，Polarion 会运行脚本来递增版本号。当保存操作完成后，他们将看到新的版本号。

脚本实现

ScriptFunction（脚本函数）是一个在用户执行特定工作流操作时，在工作项、文档或测试运行的保存操作期间调用的脚本。

可用对象

ScriptFunction 脚本可以访问以下 Polarion Open API 接口：

- `com.polarion.alm.tracker.workflow.ICallContext`（用于获取执行工作流的工作项、文档或测试运行对象。）
- `com.polarion.alm.tracker.workflow.IArguments`（用于获取脚本的自定义配置设置。）

在脚本中，您可以访问以下变量：

- `workflowContext`：ICallContext 类型的对象
- `arguments`：IArguments 类型的对象

ScriptFunction 的开始

通常，ScriptFunction 的开始是获取执行工作流脚本的对象。这是通过使用 workflowContext 对象并执行以下代码完成的：

JavaScript:

```
var baseObject = workflowContext.getTarget();
```

Groovy:

```
def baseObject = workflowContext.getTarget();
```

根据您是从工作项、文档还是测试运行执行工作流，workflowContext.getTarget() 调用返回以下对象类型： - `com.polarion.alm.tracker.model.IWorkItem` （如果从工作项执行。） - `com.polarion.alm.tracker.model.IModule` （如果从文档执行。） - `com.polarion.alm.tracker.model.ITestRun` （如果从测试运行执行。）

如果您在脚本配置（参见下一节）中使用自定义配置参数，则可以通过 arguments 对象访问它们。 例如，如果您有一个名为 versionFieldName 的自定义参数，则可以通过执行以下代码访问其值：

JavaScript:

```
// 获取在工作流中配置的版本字段名称。如果未提供值，默认值将为"version"
var versionFieldName = arguments.getAsString("versionFieldName", "version");
```

Groovy:

```
// 获取在工作流中配置的版本字段名称。如果未提供值，默认值将为"version"
def versionFieldName = arguments.getAsString("versionFieldName", "version");
```

ScriptFunction 的结束

ScriptFunction 不需要在脚本末尾进行任何特殊处理或返回值。如果您打开了任何文件句柄（参见"脚本调试"部分）或分配了任何内存，请确保关闭句柄并释放内存。

脚本配置

步骤 1：根据您希望配置的工作流类型，进入以下管理页面之一： - 工作项：Administration → Work Items → Workflow - 文档：Administration → Documents & Pages → Document Workflow - 测试运行：

Administration → Testing → Test Run Workflow

步骤 2：点击您要添加脚本函数的制品旁边的 Edit（编辑）。 步骤 3：点击您要添加脚本函数的操作旁边的 Edit（编辑）。 步骤 4：从 Functions（函数）下拉列表中选择 ScriptFunction，然后点击添加。 步骤 5：点击新创建的 Function 旁边的 Edit（编辑）。 步骤 6：定义以下必需参数： - engine：要使用的脚本引擎名称。支持 Groovy（.groovy）和 JavaScript/ECMAScript（.js）文件类型。有效值：groovy 表示 Groovy 引擎，js 表示 JavaScript/ECMAScript。 - script：scripts 文件夹中要作为条件运行的脚本名称。

例如，一个名为 incrementVersion.js 的 JavaScript。

如果您使用自定义参数，可以添加新参数，其中 Name 是自定义参数的名称，Value 是将在脚本中读取的值。 例如，假设脚本可以配置为使用存储版本号的自定义字段的 ID，您希望通过 versionFieldID 参数使其可配置。

在脚本中，以下代码用于读取参数的值：

JavaScript:

```
// 获取用于存储版本的版本自定义字段 ID。如果配置中未提供值，默认值将为"version"
var versionCustomFieldID = arguments.getAsString("versionFieldID", "version");
```

Groovy:

```
// 获取用于存储版本的版本自定义字段 ID。如果配置中未提供值，默认值将为"version"
def versionCustomFieldID = arguments.getAsString("versionFieldID", "version");
```

步骤 7：点击关闭按钮关闭 Parameters（参数）对话框，再次点击关闭 Details for Action（操作详情）对话框。 新的 Function（函数）将出现在您为其定义的操作中。

步骤 8：点击顶部的 Save（保存）。

脚本调试

ScriptFunction 没有可用的调试器。通常使用的调试方法是使用自定义日志文件，从 Polarion 日志文件夹读取这些文件。 路径：C:\Polarion\data\main\logs（Windows）和 /opt/polarion/data/main/logs（Linux）。 其思路是让脚本创建一个日志文件，用于显示脚本中变量的内容。 要使日志文件填充您的日志条目，您必须执行实现 ScriptFunction 的工作流操作，即使用工作项、文档或测试运行的 Status（状态）字段，然后点击 Save（保存）。

要为脚本创建日志文件，您可以执行：

JavaScript:

```
// 创建日志文件
var FileWriter = Java.type('java.io.FileWriter');
var outFile = new FileWriter("./logs/main/NAME_OF_YOUR_SCRIPT.log");
var BufferedWriter = Java.type('java.io.BufferedWriter');
var out = new BufferedWriter(outFile);
```

Groovy:

```
// 创建日志文件
def logFile = new File('./logs/main/NAME_OF_YOUR_SCRIPT.log').newOutputStream();
```

然后在脚本中，您可以简单地为任何您想跟踪的内容创建日志条目：

JavaScript:

```
out.write("Running NAME_OF_YOUR_SCRIPT.js script"); out.newLine(); out.flush();
```

Groovy:

```
logFile << "Running NAME_OF_YOUR_SCRIPT.groovy script \n";
```

在脚本结束时，务必始终关闭日志文件：

JavaScript:

```
// 关闭日志文件
out.close();
```

Groovy:

```
// 关闭日志文件
```

```
logFile.close();
```

throw 的使用

在 ScriptFunction 代码中可以使用 throw 语句在 Polarion UI 中生成用户消息。通过 throw 生成用户消息会停止当前的保存操作。这意味着如果触发 throw 的条件无法由用户控制，则保存操作可能永远无法完成。 通常，您应该只在开发中或非常特殊的情况下使用 throw 语句。

JavaScript:

```
throw "USER MESSAGE GENERATED BY A THROW STATEMENT";
```

Groovy:

```
throw new Exception("USER MESSAGE GENERATED BY A THROW STATEMENT");
```

以上代码在 Polarion UI 中生成以下用户消息：

脚本示例

IncrementVersion.js - 使用 2 个配置参数和调试信息

此示例递增一个字符串类型的自定义字段（参数：versionFieldID，默认值：version）。 递增值可以配置（参数：incrementValue，默认为 1.0）。 脚本还包含一个可用于调试的日志文件。 此脚本适用于工作项、文档和测试运行，因为它们都继承自同一个自定义字段接口：`com.polarion.platform.persistence.model.IHasCustomValues`（用于 getCustomField 和 setCustomField）

```
// 创建日志文件
// ---------------------
var FileWriter = Java.type('java.io.FileWriter');
var outFile = new FileWriter("./logs/main/incrementDocVersionWF.log");
var BufferedWriter = Java.type('java.io.BufferedWriter');
var out = new BufferedWriter(outFile);

// 评估传入的工作流操作参数
// versionFieldID 默认值："version"
// incrementValue 默认值："1.0"
// ------------------------------------------------
var versionCustomFieldID = arguments.getAsString("versionFieldID", "version");
out.write("Version Field Id: " + versionCustomFieldID); out.newLine(); out.flush();
var incrementValue = arguments.getAsString("incrementValue", "1.0");
out.write("Increment Value: " + incrementValue); out.newLine(); out.flush();

// 从工作流上下文构造函数实例化基础对象
// -------------------------------------------------------------------
var baseObject = workflowContext.getTarget();

// 从版本自定义字段获取自定义字段值（ID：versionCustomFieldID）
// 我们期望一个字符串类型的自定义字段
//
-------------------------------------------------------------------------------------
var oldVersion = baseObject.getCustomField(versionCustomFieldID);
var newVersion = "";
out.write("Values:"); out.newLine(); out.flush();
out.write("Old Version: " + oldVersion); out.newLine(); out.flush();

// 如果版本字段中未定义值，我们将其设置为 incrementValue 的值
// ------------------------------------------------------------------
if (oldVersion == "" || oldVersion == null || oldVersion == "NaN")
```

```
{
  newVersion = incrementValue;
  baseObject.setCustomField(versionCustomFieldID, newVersion);
  out.write("New Version: " + newVersion); out.newLine(); out.flush();
}
// 版本字段已有值，因此我们只需将其递增 incrementValue
// ------------------------------------------------------------------------------
else
{
  newVersion = parseFloat(oldVersion) + parseFloat(incrementValue);
  baseObject.setCustomField(versionCustomFieldID,
newVersion.toFixed(1).toString());
  out.write("New Version: " + newVersion.toFixed(1).toString()); out.newLine();
out.flush();
}

// 关闭日志文件
// --------------------
out.close();
```

ScriptFunction 工作流配置应为：

CreateDocumentBaseline.js

此脚本为当前正在其上执行的文档创建一个 Document Baseline（文档基线）。文档基线将与目标修订版关联。例如，如果 ScriptFunction 用于从"审核中"到"已批准"状态之间的操作，则文档基线将为与"已批准"状态关联的修订版创建。 在此示例中，我们创建了一个不属于当前正在执行工作流的文档的新对象。（因此我们需要保存基线。）

```
// 从工作流上下文构造函数实例化基础对象
// ----------------------------------------------------------------
var baseObject = workflowContext.getTarget();

// 使用 createObjectBaselineForChange 为目标修订版创建新基线
// 通过 com.polarion.alm.tracker.IBaselinesManager
//
-------------------------------------------------------------------------------
var baseline =
baseObject.getProject().getBaselinesManager().createObjectBaselineForChange(baseObj
ect);

// 设置基线的名称和描述
// 参见 com.polarion.alm.tracker.model.IBaseline
// -------------------------------------------
baseline.setName(createName());
baseline.setDescription(com.polarion.core.util.types.Text.plain(createDescription()
));

// 保存基线
// 由于我们创建了一个不属于当前文档的新对象，必须保存它
//
-------------------------------------------------------------------------------
---------
baseline.save();

// 定义所创建基线的名称
// -------------------------------------
```

```
function createName()
{
  return baseObject.getEnumerationOptionForField("status",
workflowContext.getTargetStatusId()).getName();
}

// 定义所创建基线的描述
// -------------------------------------------
function createDescription()
{
  return "Created automatically by a workflow transition";
}
```

作业脚本

Polarion 自带一个执行作业的调度器。其中许多作业是默认包含的，新的作业可以通过 Java 扩展创建。包含的作业类型之一是 script.job，它可以执行自定义的 Groovy 或 JavaScript/ECMAScript 脚本。（支持的版本请参见"脚本设置与支持的语言"部分。）

这些作业脚本可以访问所有 Polarion 数据，并执行数据修改自动化活动（当为它们设置了权限时）、以特定格式将数据导出到特定位置、发送自定义电子邮件通知等。

脚本配置

要创建 script.job，必须在 Polarion Scheduler（调度器）（全局管理 → Scheduler（调度器））中创建一个新条目。

与任何 Polarion 作业一样，可以在 XML 条目中使用以下属性来配置作业：

- name：（必需）在 Monitor（监控器）中查看时使用的作业名称。
- id：（必需）Polarion 用来查找作业的标识符。对于作业脚本，标识符必须是：script.job。
- scope：（必需）作业的范围。必须是以下之一：
- project:
- path:
- system
- cronExpression：（仅在作业需要自动执行时必需）。UNIX cron 风格的时间点规范，指定应启动作业的时间。请参阅全局管理中 Scheduler（调度器）主题的快速帮助以获取示例。
- disabled：（可选 - 默认值为"false"）。当设置为"false"时，作业将由调度器根据 cron 表达式自动执行。当设置为"true"时，作业只能通过 Monitor（监控器）手动执行。
- node：（可选）设置执行作业的集群节点。可能的值为：
- 未指定 - 调度器将选择其中一个节点
- node="*" - 将在集群中的所有节点上执行
- node="" - 仅在指定节点上执行

script.job 条目必须使用值"script.job"作为 ID。

然后，作业具有以下必需参数： - scriptName：要运行的脚本名称。（它必须位于 scripts 文件夹中。） - scriptEngine：要使用的脚本引擎名称。支持 Groovy（.groovy）和 JavaScript/ECMAScript（.js）文件类型。有效值：groovy 表示 Groovy 引擎，js 表示 JavaScript/ECMAScript。

以下是一个 Scheduler（调度器）script.job 条目示例，用于一个 JavaScript 作业，该作业每天 8:00 到 18:00 之间每 10 分钟自动从 XML 文件导入测试结果：

```
<job name="Test results importer via XML" id="script.job" cronExpression="0 0/10
8-17 * * ?" disabled="false" scope="system">
    <scriptName>TestResultsImporter.js</scriptName>
    <scriptEngine>js</scriptEngine>
</job>
```

也可以添加可由 script.job 读取的自定义属性。这些属性在作业的 `<properties>` 元素内定义。以下示例中的属性将作为名为 importFolder 的变量访问，其值为 c:\importResults。以下是一个 Groovy 脚本示例：

```
<job name="Test results importer via XML" id="script.job" cronExpression="0 0/10
8-17 * * ?" disabled="false" scope="system">
    <scriptName>TestResultsImporter.groovy</scriptName>
    <scriptEngine>groovy</scriptEngine>
    <properties>
        <importFolder>c:\importResults</importFolder>
    </properties>
</job>
```

在脚本中，自定义属性的名称成为一个变量，其内容为属性值：

JavaScript 和 Groovy

```
// 通过变量 importFolder 获取为 <importFolder> 配置的属性值
logger.info("-------------- Parameters --------------");
logger.info("The value of the parameter --importFolder-- is " + importFolder);
```

脚本实现

script.job 是一种可以通过 Polarion Monitor（监控器）主题手动执行的脚本，也可以通过 Polarion Scheduler（调度器）自动执行。 （默认在任何 Polarion 项目中可用，或通过 Administration（管理）中的 Portal（门户）主题添加。请参阅帮助中"配置门户"和"配置调度器"主题。）

可用对象

在脚本中，您将能够访问以下变量。

script.job 脚本可以通过其对应的变量访问以下 Polarion Open API 接口：

- logger：一个 com.polarion.platform.jobs.ILogger，可用于记录作业的消息。

- scope：一个 com.polarion.platform.context.IContext，表示作业的范围。
- workDir：一个指向作业工作目录的 java.io.File。
- jobUnit：运行脚本的 com.polarion.platform.jobs.IJobUnit。
- trackerService：com.polarion.alm.tracker.ITrackerService 服务，提供对跟踪相关功能和任何 Polarion 制品 API 的主要入口点。
- projectService：com.polarion.alm.projects.IProjectService 服务，提供对项目相关功能和 API 的主要入口点。
- result：一个布尔变量，可用于在作业日志文件中显示作业执行的状态。

script.job 的内容

如何使用 scope

scope 对象包含在 Scheduler（调度器）的 XML 配置条目中定义的 script.job 的范围。script.job 决定是否使用该范围。

使用 scope 的一种方式是指定 script.job 应在其上执行的项目的 projectID。 在这种情况下，您可以按如下方式读取和使用 scope 值：

JavaScript:

```
// 读取 scope 值（使用 scope 对象）
var jobScope = scope.getId().getContextName();
var tProjects = [];
if(jobScope != null)
{
    tProjects = projectService.searchProjects(jobScope);
}
```

Groovy:

```
// 读取 scope 值（使用 scope 对象）
def jobScope = scope.getId().getContextName();
def tProjects = [];
if(jobScope != null)
{
    tProjects = projectService.searchProjects(jobScope);
}
```

如何读取脚本属性

如"脚本配置"部分所述，script.job 的 XML 配置条目可以定义可从脚本读取的自定义属性。最佳实践是在脚本开头读取这些自定义属性，记录其值，并在它们不包含预期值时进行处理（例如，如果它们为空）。

如果我们有一个 script.job XML 配置定义了：

```
    <properties>
        <importFolder>c:\importResults</importFolder>
    </properties>
```

那么在 script.job 中，您可以使用 try/catch 组合。

JavaScript:

```
var sImportFolder = "";
try
{
    // 检查 importFolder 属性是否已定义，如果未定义则带错误退出作业
    logger.info("-------------- Parameters --------------");
    if( importFolder == null || importFolder.trim().length() == 0 )
    {
        throw "Error: Missing value for parameter 'importFolder'";
    }
    else
    {
        logger.info("The value of the parameter --importFolder-- is " +
importFolder);
        sImportFolder = importFolder;
    }
    if(!sImportFolder.equals(""))
    {
        // 在这里，这是作业的实际代码开始的地方
        //(...)
    }
}
```

```
catch (err)
{
    // 错误消息添加到日志中
    logger.error(err);
}
```

Groovy:

```
def sImportFolder = "";
try
{
    // 检查 importFolder 属性是否已定义，如果未定义则带错误退出作业
    logger.info("-------------- Parameters --------------");
    if( importFolder == null || importFolder.trim().length() == 0 )
    {
        throw "Error: Missing value for parameter 'importFolder'";
    }
    else
    {
        logger.info("The value of the parameter --importFolder-- is " +
importFolder);
        sImportFolder = importFolder;
    }
    if(!sImportFolder.equals(""))
    {
        // 在这里，这是作业的实际代码开始的地方
        //(...)
    }
}
catch (err)
{
    // 错误消息添加到日志中
    logger.error(err);
}
```

如何使用 workDir 变量

script.job 将作业的工作目录作为 java.io.File 对象提供。每次执行作业都会在主作业工作目录下创建一个不同的 workDir。

主作业工作目录位于： 路径：C:\Polarion\path\workspace\polarion-data\jobs\（Windows）和 /opt/polarion/workspace/polarion-data/jobs/（Linux）。

如果您需要操作作业工作目录中的文件，这会很有用。 例如：

JavaScript:

```
var workDirPath = workDir.getAbsolutePath();
logger.info("The job executes in the folder: " + workDirPath);
```

Groovy:

```
def workDirPath = workDir.getAbsolutePath();
logger.info("The job executes in the folder: " + workDirPath);
```

如何启动和提交事务

当 script.job 修改 Polarion 内容（工作项、文档、测试运行等）时，它必须使用事务（Transaction）。其思路是在需要启动和提交（也可以回滚）的事务上下文中执行修改，通过 ITransactionService（com.polarion.platform.ITransactionService）实现。

事务以手动执行 script.job 的用户身份提交，如果 script.job 由 Polarion Scheduler（调度器）自动执行，则以 polarion 用户身份提交。（请参阅"script.job 的权限"部分，确保您具有执行 script.job 的正确权限。）

例如：

JavaScript:

```
// 检索事务服务
var txService =
com.polarion.platform.core.PlatformContext.getPlatform().lookupService(com.polarion
.platform.ITransactionService.class);

// 启动新事务，因为用户可能被停用
txService.beginTx();

// 修改 Polarion 内容的代码
```

```
// 不要忘记在提交事务之前对任何被修改的 Polarion 对象调用".save()"
// (...)

// 提交事务
// endTx(false) 将提交事务
// endTx(true) 将回滚事务期间所做的更改
txService.endTx(false);
```

Groovy:

```
// 检索事务服务
def txService =
com.polarion.platform.core.PlatformContext.getPlatform().lookupService(com.polarion
.platform.ITransactionService.class);

// 启动新事务，因为用户可能被停用
txService.beginTx();

// 修改 Polarion 内容的代码
// 不要忘记在提交事务之前对任何被修改的 Polarion 对象调用".save()"
// (...)

// 提交事务
// endTx(false) 将提交事务
// endTx(true) 将回滚事务期间所做的更改
txService.endTx(false);
```

使用 doAsUser 以其他用户身份运行 script.job 的部分代码

doAsUser 方法（com.polarion.platform.security.ISecurityService）允许您以其他用户身份运行 script.job 中定义的函数。当 script.job 正在修改或创建新的 Polarion 数据并且需要由 Scheduler（调度器）自动运行（它使用 polarion 用户，该用户对 Polarion 数据只有只读访问权限）时，这会很有用。

要使用 doAsUser 以其他用户身份运行 script.job 的部分代码，您必须首先确定（或创建）一个具有正确权限级别的 Polarion 用户，以便访问和修改您希望 script.job 修改的 Polarion 数据。然后，您必须为此用户创建一个 User Account Vault（用户账户库）条目。

要创建新的 User Account Vault Entry（用户账户库条目）：

步骤 1：以具有存储库管理员权限的用户登录。 步骤 2：打开 Repository（存储库）并进入 Administration（管理）。 步骤 3：展开 User Management（用户管理）并选择 User Account Vault（用户账户库）。 将显示现有 Key（密钥）的 User Account Vault（用户账户库）表格。

步骤 4：Add（添加）或 Remove（删除）一个 Key（密钥）： - 添加密钥：输入任何 Key 名称、Polarion User Name（将用于访问数据的用户的用户名）、其 Password（密码），然后重新输入 Password（密码），然后点击添加。

步骤 5：点击 Save（保存）。

在您的 script.job 中，应该创建一个将以其他用户身份运行的专用函数。该函数将包含需要访问目标 Polarion 数据的代码。例如，在函数中，您可能会启动一个事务、修改或添加 Polarion 内容，然后提交事务。然后，使用 doAsUser，您将告诉 Polarion 以用户（及其所有权限）的上下文通过您添加到 User Account Vault（用户账户库）的密钥来执行该函数。

例如：

JavaScript (使用 GraalVM):

```
// 此函数需要由具有 Polarion 存储库读/写访问权限的用户执行
function MyFunctionWithRW()
{
    logger.info("MyFunctionWithRW - Code doing a modification on the Polarion data
(Requires RW Access)")
}

// 使用添加到 Polarion Account Vault 并用密钥"rProject.key"标识的具有 RW 访问权限的用户登录
var securityService = trackerService.getDataService().getSecurityService();
var userWithRWAccess = securityService.loginUserFromVault("rProject.key", "");

// 创建一个 Privileged Action（特权操作）对象，将由 doAsUser 函数与我们的 RW 访问用户一起运行
var privilegedAction = Java.extend(Java.type('java.security.PrivilegedAction'));
var privilegedActionImpl = new privilegedAction({ run: function () {
{
    logger.info('User used by doAsUser to execute this call is: ' +
securityService.getCurrentSubject().toString());

    // 我们对需要 RW 访问权限的函数的调用
    MyFunctionWithRW();
```

```
    return null;
}});

// 使用 doAsUser 代表 userWithRWAccess 中的用户调用 privilegedActionImpl 函数
securityService.doAsUser(userWithRWAccess, privilegedActionImpl);
```

JavaScript (使用 Nashorn):

```
// 此函数需要由具有 Polarion 存储库读/写访问权限的用户执行
function MyFunctionWithRW()
{
    logger.info("MyFunctionWithRW - Code doing a modification on the Polarion data
(Requires RW Access)")
}

// 使用添加到 Polarion Account Vault 并用密钥"rProject.key"标识的具有 RW 访问权限的用户登录
var securityService = trackerService.getDataService().getSecurityService();
var userWithRWAccess = securityService.loginUserFromVault("rProject.key", "");

// 创建一个 Privileged Action（特权操作）对象，将由 doAsUser 函数与我们的 RW 访问用户一起运行
var privilegedActionImpl = new Object();
privilegedActionImpl.run = function ()
{
    logger.info('User used by doAsUser to execute this call is: ' +
securityService.getCurrentSubject().toString());

    // 我们对需要 RW 访问权限的函数的调用
    MyFunctionWithRW();

    return null;
}

// 使用 doAsUser 代表 userWithRWAccess 中的用户调用 privilegedActionImpl 函数
securityService.doAsUser(userWithRWAccess, new
java.security.PrivilegedAction(privilegedActionImpl));
```

Groovy:

```
// 此函数必须由具有 Polarion 存储库读/写访问权限的用户执行
void MyFunctionWithRW()
{
    logger.info("MyFunctionWithRW - Code doing a modification on the Polarion data
(Requires RW Access)")
}

// 使用添加到 Polarion Account Vault 并用密钥"rProject.key"标识的具有 RW 访问权限的用户登录
def securityService = trackerService.getDataService().getSecurityService();
def userWithRWAccess = securityService.loginUserFromVault("rProject.key", "");

// 使用 doAsUser 代表 userWithRWAccess 中的用户调用 MyFunctionWithRW() 函数
def foo = securityService.doAsUser(
    userWithRWAccess,
    {
        logger.info('User used by doAsUser to execute this call is: ' +
securityService.getCurrentSubject().toString());
        MyFunctionWithRW()
    } as java.security.PrivilegedAction
);
```

脚本执行

> 警告：根据作业的范围和类型，它可能会消耗系统资源并显著降低其他用户的系统速度。在决定明确运行作业时，请牢记这一点。

手动执行 script.job

Monitor（监控器）主题提供有关当前正在运行和已调度的作业的信息。该主题在项目内工作或在全局（存储库）范围内工作时可用。

从该主题中，用户可以通过选中其关联的复选框来选择任何可用的作业，然后点击 Execute now（立即执行）。

注意：作业只有在添加到 Scheduler（调度器）后才会在 Monitor（监控器）中列出。（请参阅"脚本配置"部分。）

自动执行 script.job

script.job 可以由 Polarion 在任何时间间隔自动调度运行。详情请参阅"脚本配置"部分。

script.job 的权限

当通过 Polarion Monitor（监控器）主题手动执行 script.job 时，script.job 以执行它的用户身份执行。（它只能访问用户有权限访问的数据。）这很重要，因为 script.job 可能是由具有完全管理员权限的用户创建和测试的，但执行它的用户可能没有访问或修改数据的必要权限，导致脚本意外失败。

script.job 可以通过 Polarion Scheduler（调度器）自动执行。当这种情况发生时，Polarion 以 polarion 用户身份执行脚本。此系统用户对所有 Polarion 数据具有只读访问权限。这很重要，因为如果 script.job 预期修改任何数据，则通过 Scheduler（调度器）的自动执行将失败。最佳实践是使用 doAsUser() 方法实现 script.job 中需要对数据进行写访问的部分，如"使用 doAsUser 以其他用户身份运行 script.job 的部分代码"部分所述。

脚本调试

script.job 没有可用的调试器。通常使用的调试方法是使用作业日志记录器（job logger）。作业日志记录器允许您写入与正在运行的作业关联的作业日志，可以通过点击与作业关联的日志链接从作业监控器访问。

其思路是使用作业日志来填充调试脚本所需的任何信息。每次作业手动执行（通过 Monitor（监控器））或通过 Scheduler（调度器）自动执行时，都会生成一个作业日志。要使用日志记录器，您需要使用 logger 对象，可以使用不同的日志消息级别：

JavaScript 和 Groovy

```
logger.info("This will generate an Info type message in the job log");
logger.error("This will generate an Error type message in the job log");
logger.warn("This will generate an Warn type message in the job log");
logger.fatal("This will generate an Fatal type message in the job log");
```

这些代码在日志中生成以下条目：

```
2021-12-15 15:53:12,943 [Worker-0: My Script Job | u:admin | job: script.job] INFO
root  - This will generate an Info type message in the job log
2021-12-15 15:53:12,950 [Worker-0: My Script Job | u:admin | job: script.job] ERROR
root  - This will generate an Error type message in the job log
2021-12-15 15:53:12,957 [Worker-0: My Script Job | u:admin | job: script.job] WARN
root  - This will generate an Warn type message in the job log
2021-12-15 15:53:12,963 [Worker-0: My Script Job | u:admin | job: script.job] FATAL
root  - This will generate an Fatal type message in the job log
```

脚本示例

emailDigestJob.js

此作业示例向存储库中所有用户（未关闭通知的用户）发送邮件，这些用户被邀请签署文档但尚未签署。 调度器配置应如下所示：（此处配置为每周每个工作日的早上 6 点执行。）

```
<job name="Email Digest" id="script.job" cronExpression="0 00 6 ? * MON-FRI"
disabled="false" scope="system">
    <scriptName>emailDigestJob.js</scriptName>
    <scriptEngine>js</scriptEngine>
    <properties>
        <subjectPrefix>[MY POLARION SERVER]</subjectPrefix>
        <senderAddress>any_email@polarion.com</senderAddress>
    </properties>
</job>
```

> 警告：要使此作业正常工作，您的 Polarion 服务器必须正确配置 SMTP 服务器并能够发送电子邮件通知。详情请参阅您操作系统的安装文档。

```
/**
 **  Script:      emailDigestJob.js
 **
 **  Purpose:     This job will get a list of all users in the repository and
notify them if:
 **                 - Documents are awaiting their signatures
 **
 **  Polarion Job Parameters - these variables, while never defined are created by
the script extension
 **     senderAddress : Senders (From) Address to send the send from
 **     subjectPrefix : Subject prefix for all outgoing emails.
 **
 **   Example Configuration
 **       This configuration will execute the job every working day in the week at
6am.
 **       <job cronExpression="0 00 6 ? * MON-FRI" disabled="false" id="script.job"
name="Email Digest" scope="system">
 **         <scriptName>emailDigestJob.js</scriptName>
 **         <scriptEngine>js</scriptEngine>
 **         <properties>
 **           <subjectPrefix>[Polarion Server DEV]</subjectPrefix>
 **           <senderAddress>any_email@polarion.com</senderAddress>
 **         </properties>
 **       </job>
**/
logger.info(" ");
logger.info("-------------- Parameters --------------");
logger.info("subjectPrefix        = " + subjectPrefix);
logger.info("senderAddress        = " + senderAddress);

// Function sendEmail
// subject: Takes a string to use as the subject line of the email to be sent
// senderAddress: Takes a string to use as the sender address of the email to be
sent
// emails: an array of emails addresses as strings --> string emails
// message: a string representing the HTML body of the emails
//
-------------------------------------------------------------------------------
function sendEmail(subject, senderAddress, emails, message)
{
  // Sending an email notification by using the Polarion Announcement Service
  // -------------------------------------------------------------------
  var receiver = [emails];
  var headerString = "From: "+ senderAddress + "\n";
  headerString = headerString + "To: " + emails + "\n";
  headerString = headerString + "Subject: " + subject + "\n\n";
```

```
  var announcerSrv =
com.polarion.platform.core.PlatformContext.getPlatform().lookupService(com.polarion
.platform.announce.IAnnouncerService.class);
  var errMsg = "";
  if (announcerSrv !== 'undefined')
  {
    var Announcement = Java.type('com.polarion.platform.announce.Announcement');
    var announcement = new Announcement();
    announcement.setSender(senderAddress);
    announcement.setReceivers(receiver);
    announcement.setContentType("text/html");
    announcement.setSubject(subject);
    announcement.setContent(message);
    try
```

```
    {
       announcerSrv.sendAnnouncement("smtp", announcement);
    }
    catch(ex)
    {
       errMsg = "Could not send email notification. Please, check mail settings for
Polarion. \n\n" + ex;
       logger.error(errMsg);
    }
  }
  else
  {
    logger.info("We are not able to get the Announcer Service");
  }
  logger.info("\n---------------------------------- eMail notification
---------------------------------- \n");
  logger.info(headerString + message + "\n\n");
  if(errMsg != "")
  {
    logger.error("Error Message: " + errMsg + "\n\n");
  }
}

// Constructor EmailMessage
// subject: Takes a string that will be the subject of the email message you are
writing
// addresses: Takes an array of strings that are the email address to send this
email to
// body: Takes a string that will be sent as the email body. Do not include
beginning and ending <html> tags
//
// void addBodyLine(string): adds a new line to the body of the email
// string getBodyHTML(): returns the body of the email wrapped in <html> tags
// string createHTMLLink(string, string): returns a valid HTML link pointing to the
URL with text text
//
-------------------------------------------------------------------------------------
-----------------------
function EmailMessage(subject, addresses, body)
{
  this.subject = subject;
  this.addresses = addresses;
  this.body = body;
  this.addBodyLine = function(newLine) {
  this.body = this.body + "<br>\n" + newLine;
  }
```

```
  this.addBodyLineNoBR = function(newLine) {
  this.body = this.body + "\n" + newLine;
  }
  this.getBodyHTML = function() {
  return "<html>\n" + this.body + "\n</html>";
  }
}
function createHTMLLink(url, text)
{
    return "<a href=\"" + url + "\">" + text + "</a>"
}

// -------------------------------   MAIN CODE -------------------------------- //
try
{
  var dataService = trackerService.getDataService();
  var projectsService = trackerService.getProjectsService();
  var myUsers = projectsService.getUsers()
  for (i = 0; i < myUsers.size(); i++)
  {
    // Getting info on the users
    // --------------------------
    var myUser = myUsers.get(i);
    var myUserId = myUser.getId();
    var isUserDisabled = myUser.isDisabled();
    var myUserEmailAddress = "" + myUser.getEmail() + "";
    var isMyNotificationDisabled = myUser.hasDisabledNotifications();
    // Check if the user is disabled or if user notification is disabled or user
mail id is null
    //
-------------------------------------------------------------------------------
------
    if( isUserDisabled || isMyNotificationDisabled || myUserEmailAddress == "null"
|| myUserEmailAddress == "")
    {
```

```
      if(isUserDisabled)
      {
        // Skip processing this user as his account is disabled
        // -----------------------------------------------------
        logger.info("Skip disabled user: " + myUserId + " email ID :" +
myUserEmailAddress + "\n");
      }
      else if(isMyNotificationDisabled)
      {
        // Skip any further processing for this user ( as users email notifications
are disabled)
        //
-------------------------------------------------------------------------------------
---
        logger.info("skip processing user: " + myUserId + " Disable Notification
status : " + isMyNotificationDisabled + "\n");
      }
      else if(myUserEmailAddress == "null" || myUserEmailAddress == "")
      {
        // Skip any further processing for this user (as the user email id is not
set)
        //
-------------------------------------------------------------------------
        logger.info("skip processing user: " + myUserId + " email ID is NOT SET,
email ID: " + myUserEmailAddress + "\n");
      }
      continue;
    }

    // Query to find all documents a specific user has to sign
    // -------------------------------------------------------
    var documentsToSign = dataService.searchInstances("Module",
"signatures:invited=" + myUserId, null);

    // No Documents to sign
    // --------------------
    if (documentsToSign.size() == 0)
    {
      logger.info("No signatures for: " + myUserId + "\n");
    }
    else
    {
      logger.info("Preparing for user: " + myUserId + "\n");

      // Creating the email message with the subject, the user email address and
the start of the email
```

```
      //
-------------------------------------------------------------------------------------
-----------
      var message = new EmailMessage(subjectPrefix + " Polarion Awaiting Document
Signatures", myUserEmailAddress, "<head><title>Polarion Awaiting Signatures
Digest</title></head><h2>Awaiting Your Document Signatures</h2>");

      // If the user is assigned to sign some Documents, then a table will be
created in the email and every Document will be listed
      //
-------------------------------------------------------------------------------------
----------------------------------------
      if (documentsToSign.size() > 0)
      {
        message.addBodyLineNoBR("<table cellpadding=4 ><tr
bgcolor=\"#E1F0FE\"><th>Title</th><th>Space</th><th>Project</th></tr>");
        for (j = 0; j < documentsToSign.size(); j++)
        {
          var myDoc = documentsToSign.get(j);
          var pId = myDoc.getProject().getId();
          myFullDocName = myDoc.getModuleFolder() + "/" + myDoc.getModuleName();
          message.addBodyLineNoBR('<tr><td>' + myFullDocName + '</td><td>' +
myDoc.getModuleFolder() + '</td><td>' + myDoc.getProject().getName() + '</td></
tr>');
        }
        message.addBodyLineNoBR("</table>");
      }
      else
      {
        message.addBodyLineNoBR("<h4>No Awaiting Signature</h4>");
      }

      // Uncomment this to print the message body in the logger (for debugging
purpose)
      //
-------------------------------------------------------------------------
      // logger.info(message.getBodyHTML());

      // Send email (you can comment this line to not send any email during
debugging sessions)
      //
-------------------------------------------------------------------------------------
---
      sendEmail(message.subject, senderAddress, message.addresses,
message.getBodyHTML());
    }
  }
```

```
  result = true;  // This will add this message in the log file -> Script returned:
true
}
catch (e)
{
  logger.error("Job generated an error:" + e);
```

```
  result = false; // This will add this message in the log file -> Script returned:
false
}
```

FAQ（常见问题）

1- 在哪里可以获得脚本和 Polarion API 的帮助?

回答：向其他 Polarion 用户和专家提问并获得答案的最佳地点是 Polarion 社区网站。

2- 在 ScriptCondition 中是否需要对执行工作流的对象（workflowContext.getTarget()）调用 .save()?

回答：您绝不应在 ScriptCondition 中对执行工作流的对象调用 .save()。（它不是一种用于修改 Polarion 数据的脚本类型。）

3- 在 ScriptFunction 中是否需要对执行工作流的对象（workflowContext.getTarget()）调用 .save()?

回答：您绝不应在 ScriptFunction 中对执行工作流的对象调用 .save()，因为该脚本已经在 Polarion 保存基础对象时执行。如果您的 ScriptFunction 修改了另一个对象（例如，链接到当前正在执行工作流的工作项的另一个工作项），那么是的，应在此另一个对象上调用 .save()。

4- 我已经更新了 script.job 的 Scheduler（调度器）条目，但当我在 Monitor（监控器）中尝试手动运行它时，它没有获取到我的修改。

回答：如果您正在更新作业的 Scheduler（调度器）条目，请确保刷新 Monitor（监控器）页面以获取更改。

5- 我的 script.job 在通过调度器自动执行时失败，但手动运行时没有失败。

回答：这意味着作业在通过 Scheduler（调度器）自动执行时没有对 Polarion 数据的相同访问权限，因为它使用 polarion 用户（该用户对数据只有只读访问权限）。要成功执行，您必须使用 doAsUser 方法。 详情请参阅"使用 doAsUser 以其他用户身份运行 script.job 的部分代码"部分。

6- 使用 GraalVM 引擎执行 JavaScript 时，出现此类错误：TypeError: (...) : Message not supported。

回答：此错误是由于使用完全限定的 Java 类名声明变量所致。 例如：

```
var currentDate = new java.util.Date();
```

使用 GraalVM，必须使用 Java.type(typename) 例如：

```
var Date = Java.type('java.util.Date');
var currentDate = new Date();
```

7- 使用 GraalVM 引擎执行 JavaScript 时，出现此类错误：ReferenceError: importPackage is not defined。

回答：此错误是由于使用了不受支持的"importPackage"语句所致。（可能来自为 Rhino 编写的脚本。） 例如：

```
importPackage(com.polarion.platform);
importPackage(com.polarion.platform.core);
importPackage(com.polarion.platform.context);
```

要修复它，首先确保您需要这个包。如果不需要，删除它。 如果您需要它，例如：

```
importPackage(java.io);
(...)
var outFile = new FileWriter("./logs/main/NAME_OF_YOUR_SCRIPT.log");
```

使用 GraalVM，使用 Java.type(typename) 访问特定类：

```
var FileWriter = Java.type('java.io.FileWriter');
var outFile = new FileWriter("./logs/main/NAME_OF_YOUR_SCRIPT.log");
```

8- 使用 GraalVM 引擎执行 JavaScript 时，出现此类错误：Illegal char <:> at index X: nashorn:mozilla_compat.js

回答：此错误是由于使用了不受支持的"load("nashorn:mozilla_compat.js");"语句所致。（可能来自从 Rhino 转换到 Nashorn 的脚本。） 要修复它，完全删除该语句。您可能需要重写包含包的方式。请参阅"FAQ 问题 #6 和 #7"以获取更多信息。

9- 使用 GraalVM 引擎执行 JavaScript 时，出现此类错误：ReferenceError: JavaImporter is not defined。

回答：此错误是由于使用了不受支持的"JavaImporter"语句所致。（可能来自为 Nashorn 编写的脚本。） 例如：

```
var JavaPackages = new JavaImporter(java.io,
                                    java.text,
                                    java.lang

                                );
```

使用 GraalVM，应为要访问的每个特定类型的类使用 Java.type(typename) 定义一个变量。

如果您需要它，例如：

```
var JavaPackages = new JavaImporter(java.io);
with( JavaPackages )
{
   var outFile = new FileWriter("./logs/main/NAME_OF_YOUR_SCRIPT.log");
}
```

使用 GraalVM，应写为：

```
var FileWriter = Java.type('java.io.FileWriter');
var outFile = new FileWriter("./logs/main/NAME_OF_YOUR_SCRIPT.log");
```

10- 使用 GraalVM 引擎执行 JavaScript 时，无法在字符串上使用 java.lang.String 方法。

回答：GraalVM 将字符串视为 JavaScript 字符串，而不是 Java 字符串。 例如：

```
var MyString = "This is my String";
if(MyString.equals("This is my String")
{
  // The strings are equal!
}
```

将生成此错误：TypeError: MyString.equals is not a function 使用 GraalVM，应写为：

```
var MyString = "This is my String";
if(MyString.valueOf() == "This is my String")
{
  // The strings are equal!
}
```

JavaScript 字符串列表可以在以下位置找到：https://www.w3schools.com/jsref/jsref_obj_string.asp Nashorn 脚本引擎默认将字符串视为 java.lang.String。因此，为 Nashorn 编写的使用字符串的脚本可能需要适配 GraalVM。

11- 使用 GraalVM 引擎执行 JavaScript 时，出现此类错误：Cannot convert 'XX.YY'(...) to Java type 'float': Invalid or lossy primitive coercion。

回答：使用 GraalVM 引擎设置 Float 类型的字段需要显式完成。 例如：

```
var priority = 7.505;
workItem.setPriority(workItem.createPriorityOpt(priority.toFixed(2)));
```

将生成此错误：Cannot convert '7.505' (...) to Java type 'float': Invalid or lossy primitive coercion. 使用 GraalVM，应写为：

```
var priority = 7.505;
var Float = Java.type('java.lang.Float');
workItem.setPriority(workItem.createPriorityOpt(Float.valueOf(priority.toFixed(2)))
);
```

12- 使用 GraalVM 引擎执行 JavaScript 时，出现此类错误：'X.Y' is not instance of java.lang.Float。

回答：GraalVM 处理从 String 到 Float 的转换与之前的 JavaScript 引擎不同。 为此，您可以使用以下方法：

```
var floatValue = "2.0";
workItem.setCustomField("myFloatFieldID", parseFloat(floatValue));
```

或者使用以下方法：

```
var Float = Java.type('java.lang.Float');
var floatValue = new Float('2.0');
workItem.setCustomField("myFloatFieldID", floatValue);
```

13- 执行 Groovy 脚本时，在尝试检索当前 Groovy 版本时生成错误。

回答：此错误是由于使用了不受支持的 getVersion() 调用所致，可能来自为 v1.5.7 编写的 Groovy 脚本。 例如：

```
org.codehaus.groovy.runtime.InvokerHelper.getVersion();
```

使用 Groovy 2.4.21，应写为：

```
GroovySystem.getVersion();
```